GLA UNIVERSITY, GREATER NOIDA

Greater Noida - 201310, Uttar Pradesh, India

STROKE-LEVEL HANDWRITING SYNTHESIS: A VECTOR-OUTPUT GENERATIVE MODEL FOR COMPOSABLE HANDWRITTEN DOCUMENTS

SOFTWARE REQUIREMENT SPECIFICATION

Version 1.1

BACHELOR OF TECHNOLOGY
in
COMPUTER SCIENCE & ENGINEERING - AI

Submitted By

AGRIIM RASTOGI

Roll No.: 22515500004 | Section: ND

Under the Supervision of
Palash Sir / Vansh Sir
Faculty Mentors, GLA University

Reviewed By
Gautam Mukherjee
GLA University, Greater Noida

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING
GLA UNIVERSITY, GREATER NOIDA - 201310
SEPTEMBER, 2026

Revision History

Version	Date	Change	Reason
1.0	05-09-2026	Initial baseline requirements specification.	First institutional submission.
1.1	11-09-2026	Detailed system architecture, sprint date ranges, entity schemas, and ER diagram.	Feedback from Faculty Reviewer (Gautam Mukherjee).

1. Introduction

1.1 Purpose

Specifies software requirements for Stroke-Level Handwriting Synthesis v1.1, an offline deep-learning system generating human-like vector stroke trajectories from digital text. Defines functional, data, and quality constraints for verification.

1.2 Scope

Accepts English ASCII strings, sampling temperatures, and style seeds to synthesize resolution-independent SVG vector handwriting for offline document generation. Excludes OCR, scanned image transcription, and direct CNC pen-plotter motor controls.

1.3 Definitions

Term	Definition
Vector Stroke	Time-ordered 2D coordinate sequence (x, y) between pen-down and pen-up events.
Online Handwriting	Handwriting recorded as dynamic temporal trajectories (dx, dy, dt, pen_up), distinct from static 2D bitmaps.
CSR Transcription	Computer Supported Recognition ground-truth text of human writing (including writer typos), distinct from printed prompts.
X-Height (h_x)	Vertical height of lowercase letters without ascenders/descenders (e.g., 'x'), used as normalization unit.
Airborne Jump	Pen-tip spatial transition trajectory above paper between consecutive strokes.

2. Overall Description

2.1 User Classes

User Class	Who They Are	What They May Do
End-User / Creator	Student, educator, or creator generating notes.	Input text, tune temperature/style seeds, preview handwriting, export standalone SVG files.
ML Developer	Practitioner maintaining or extending the pipeline.	Execute IAM preprocessing, train sequence models, run test suites, benchmark position drift.

2.2 Assumptions and Dependencies

A1. Environment: Local execution on Python 3.13, PyTorch 2.6+, CUDA 12.6, RTX 3050 GPU (6 GB VRAM); CPU fallback supported.

A2. Corpus: Local offline availability of official IAM-OnDB archives in 'datas/raw/iam-ondb/' ('ascii/', 'lineStrokes/', 'original/').

A3. Privacy: Zero external cloud dependencies; complete local data confidentiality.

2.3 Constraints

C1. Licensing: IAM-OnDB is licensed strictly for non-commercial academic research.

C2. Hardware: Batch sizes bounded by 6.0 GB GPU VRAM, requiring sequence compression and ragged tensor packaging.

C3. Digitizer: Whiteboard sensor records at ~100 Hz; requires arc-length resampling to eliminate velocity-dependent point clusters.

3. Data Source and Baseline

Empirical parameters and baseline benchmark scores computed directly from inspecting the IAM-OnDB corpus:

Item	Value	Required now?
Dataset name	IAM On-Line Handwriting Database (IAM-OnDB)	Yes
Source and URL	University of Bern, FKI (https://fki.tic.heia-fr.ch/databases/iam-on-line-handwriting-database)	Yes
Licence / terms of use	Free for non-commercial academic research and educational evaluation	Yes
Number of rows and columns	12,178 raw lines across 1,907 forms (3,731,052 points); ragged (N, 4) arrays [dx, dy, pen_up, dt]	Yes
Prediction target	Next-step coordinate deltas (dx, dy, dt) and binary pen_up conditioned on text	Yes
Task type	Multi-class discrete token classification (dx, dy, dt) and binary classification (pen_up)	Yes
Class balance	Pen-down drawing steps: 92.1%; Pen-up airborne jumps: 7.9%	Yes
Primary evaluation metric	Relative Position Drift Error (E_drift measured relative to line x-height)	Yes
Why that metric	Pen strokes accumulate error via cumsum; per-step loss hides cumulative baseline drift and word collision	Yes
Level 0 baseline and score	Uniform linear interpolation between word bounding boxes achieves E_drift = 1.28 (illegible)	Yes
Known data quality problems	177 forms missing strokes, 2 line-count mismatches, ~0.25% non-monotonic timestamps (-746s), 10.8% missing writer ID	Yes
Level 1 baseline and score	Graves (2013) 3-layer LSTM with bivariate Gaussian MDN achieving E_drift = 0.22	Later
Target for this release	Dual-mode discrete sequence model targeting median E_drift <= 0.05 (p95 <= 0.08)	Later

4. System Architecture

The architecture decouples offline corpus preprocessing/training from real-time vector inference and rendering:

Figure 1A: Offline Data Preparation & Model Training Subsystem

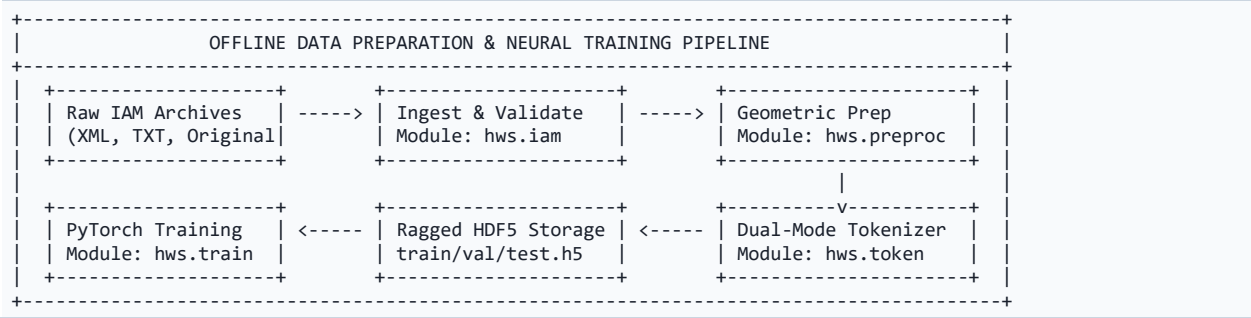
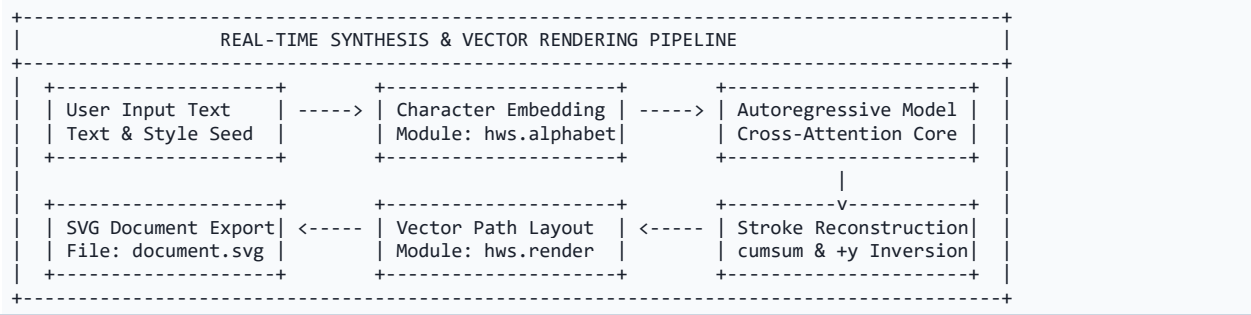


Figure 1B: Real-Time Text-to-Stroke Inference & Vector Rendering Subsystem



4.1 Core System Components

Component & Module	Input	Output	Architectural Responsibility
Corpus Ingestion (hws.iam)	Raw IAM XML, CSR text, original XML.	RawLine objects (N, 4).	Parses XML captures, validates text-stroke parity, flips y-axis (+y=up), extracts writer IDs.
Geometry Preprocessing (hws.preprocess)	Raw points (N, 4) from hws.iam.	Resampled offsets (dx, dy, dt, pen_up).	Executes arc-length resampling (1.84x compression), computes deltas, normalizes by x-height.
Character Vocabulary (hws.alphabet)	Latin ASCII character strings.	Integer token sequence (1D).	Maintains immutable 81-char vocab with <pad> and <unk> tokens, preventing index-zero collision.
Dual-Mode Discretizer (hws.tokenizer)	Continuous offsets & pen_up.	Discrete tokens (0 to N_bins - 1).	Quantizes moves into linear bins and heavy-tailed airborne jumps into uniform asinh bins.
Neural Sequence Model (hws.model)	Char tokens & prior stroke tokens.	Softmax logit probabilities.	Autoregressively predicts next-step stroke token distributions conditioned via cross-attention.
Coordinate Reconstruction (hws.preprocess)	Predicted token indices.	Absolute Cartesian arrays (x, y).	De-quantizes tokens, computes cumulative sum (cumsum), and arranges multi-line baselines.
Vector Rendering Engine (hws.render)	Cartesian coordinate arrays.	Valid SVG XML documents.	Single source of truth for converting points into SVG paths ('M/L'), reversing y to SVG standard.

5. Functional Requirements

5.1 Data Ingestion and Preprocessing

FR-1.1 Parse IAM stroke XML files into (N, 4) arrays of x, y, pen_up, and elapsed time t.

FR-1.2 Reject input forms with mismatched stroke file counts and CSR transcription lines, returning count error.

FR-1.3 Invert whiteboard sensor y-coordinates at parse time so positive y increases upward across internal tensors.

FR-1.4 Resample stroke trajectories at uniform spatial arc-length intervals determined by step factor.

FR-1.5 Normalize horizontal and vertical coordinate displacements by dividing by estimated line x-height.

5.2 Model-Backed Generation

FR-2.1 Accept strings within 81-character vocabulary; reject unsupported characters with positional error message.

FR-2.2 Return time-ordered sequence of coordinate offsets (dx, dy, dt, pen_up) for each validated text prompt.

FR-2.3 Allow adjustment of sampling temperature between 0.1 and 2.0 to control stroke curvature and legibility.

FR-2.4 Fall back to rendering pre-tokenized Level 0 baseline glyphs with warning banner on generation failure.

FR-2.5 Support selectable writer style seeds to modulate handwriting slant, speed, and ligature spacing.

6. Non-Functional Requirements

NFR-1 (Latency): Synthesize a 30-character line within 1.5 seconds on an NVIDIA RTX 3050 Laptop GPU.

NFR-2 (Privacy): Execute entirely on local host without transmitting text prompts or strokes over network.

NFR-3 (Auditability): Log preprocessing runs to timestamped JSON recording skipped files, drift percentiles, and vocabulary sizes.

NFR-4 (Resource): Compile all 12,178 corpus lines into HDF5 in < 180 seconds consuming < 2.0 GB system RAM.

7. Build Plan

7.1 Minimum Viable Product

The MVP represents the smallest self-contained version capable of demonstrating end-to-end data processing and vector stroke synthesis:

FR / NFR ID	Requirement (short)	Why this one is in the MVP
FR-1.1	Parse raw stroke XML files	Mandatory data source; without raw trajectories no training data exists.
FR-1.2	Reject mismatched form counts	Prevents misaligned text-stroke pairs from corrupting model datasets.
FR-1.4	Spatial arc-length resampling	Compresses sequences 1.84x, making neural training feasible within 6 GB VRAM.
FR-2.1	Validate text against vocabulary	Guarantees sequence model receives only valid character token embeddings.
FR-2.2	Return vector stroke sequences	Core capability required to synthesize visible handwriting trajectories.
NFR-1	Line generation under 1.5s	Maintains responsive interactive user experience during note synthesis.
NFR-2	100% offline local execution	Guarantees complete document confidentiality without cloud APIs.

Deliberately not in MVP: Few-shot style conditioning on user-uploaded handwriting samples (deferred to Sprint III).

7.2 Build Order & Sprint Milestones

Project implementation schedule across sequential development sprints with concrete milestones and dates:

Sprint Stage	Date Range	Target Scope	Milestone Deliverable & Evidence
Sprint I	01 Sep 2026 - 27 Sep 2026	FR-1.1 to FR-1.5	Data ingestion, arc-length resampling; inspect_data.py displays SVG samples.
Sprint II	28 Sep 2026 - 17 Oct 2026	FR-2.1 to FR-2.3, NFR-1	Autoregressive model & CLI; synthesizes single-line SVG in < 1.5 seconds.
Sprint III	18 Oct 2026 - 07 Nov 2026	FR-2.4, FR-2.5, NFR-2	Style conditioning & multi-line layout; generates multi-line documents.
Testing	08 Nov 2026 - 17 Nov 2026	All FRs, NFR-3, NFR-4	58 unit tests pass; zero-regression log; reconstruction drift p95 < 0.08.
Deployment	18 Nov 2026 - 30 Nov 2026	All FRs and NFRs	Packaged desktop distribution, user manual, and verified clean terminal run.
Final Viva	01 Dec 2026 - 10 Dec 2026	Full Release	Institutional project demonstration, final report submission, and oral viva.

8. Data Requirements

8.1 Entities

Five persistent entities model the data lifecycle from text input to vector rendering:

Entity 1: TextPrompt -- Captures input digital text, temperature, and line width formatting.

Field Name	Data Type	Constraints & Validation Rules
id	UUID	Primary key; uniquely identifies each request.
raw_content	Text	Raw input string; length 1 to 2,000 characters; not null.
cleaned_content	Text	Sanitized ASCII string restricted to 81-character vocabulary.
sampling_temp	Float	Sampling temperature in range [0.1, 2.0]; default 1.0.
target_width_mm	Float	Target horizontal line width in mm; positive; default 180.0.
created_at	Timestamp	System-generated UTC creation timestamp; not null.

Entity 2: WriterProfile -- Represents individual handwriting stylistic characteristics and bias embeddings.

Field Name	Data Type	Constraints & Validation Rules
id	UUID	Primary key; unique internal profile identifier.
writer_id	Integer	Unique IAM volunteer writer ID; unique index; nullable for anonymous.
style_seed	Integer	Seed initializing autoregressive style embedding; non-negative.
mean_slant_deg	Float	Mean baseline slant angle in degrees; range [-45.0, +45.0].
mean_velocity	Float	Mean pen-tip speed in sensor units/sec; positive float.
style_label	String(64)	Descriptive tag (e.g., 'Cursive-Slanted', 'Upright-Print'); not null.

Entity 3: StrokeSequence -- Continuous, time-ordered sequence of pen coordinates for one text line.

Field Name	Data Type	Constraints & Validation Rules
id	UUID	Primary key; uniquely identifies each stroke trajectory.
prompt_id	UUID	FK referencing TextPrompt.id; nullable for raw IAM training lines.
writer_id	Integer	FK referencing WriterProfile.writer_id; nullable for anonymous lines.
form_id	String(16)	Parent IAM form ID (e.g., 'a01-000u'); nullable for synthetic lines.
line_index	Integer	1-based line index within document or form; positive integer.
raw_text	Text	CSR transcription string corresponding to trajectory; non-empty.
point_count	Integer	Total coordinate points in trajectory; must be >= 10.
stroke_count	Integer	Total pen-down segments (number of pen lifts + 1); positive integer.
duration_sec	Float	Total wall-clock duration of stroke writing in seconds; non-negative.
created_at	Timestamp	System-generated UTC creation timestamp; not null.

Entity 4: QuantizedTokenSequence -- Discrete integer token representation of stroke trajectory used by neural model.

Field Name	Data Type	Constraints & Validation Rules
id	UUID	Primary key; uniquely identifies each tokenized sequence.
stroke_seq_id	UUID	FK referencing StrokeSequence.id; unique constraint (strict 1:1).
dx_tokens	Binary	Contiguous int16 discrete token IDs for horizontal displacements.
dy_tokens	Binary	Contiguous int16 discrete token IDs for vertical displacements.
dt_tokens	Binary	Contiguous int16 discrete token IDs for time deltas.
pen_up_flags	Binary	Contiguous uint8 binary flags indicating pen-lift state after each step.
token_count	Integer	Total token count; must exactly equal parent StrokeSequence.point_count.

Entity 5: ExportedSVGDocument -- Finalized resolution-independent vector graphic file saved to disk.

Field Name	Data Type	Constraints & Validation Rules
id	UUID	Primary key; uniquely identifies each exported document.
prompt_id	UUID	FK referencing TextPrompt.id; not null.
file_path	String(256)	Absolute local filesystem path where SVG is saved; unique index.
viewbox_width	Float	SVG viewBox horizontal width in pixels; positive float.
viewbox_height	Float	SVG viewBox vertical height in pixels; positive float.
line_count	Integer	Total handwritten lines composed in document; positive integer.
file_size_bytes	Integer	Size of exported SVG file on disk in bytes; positive integer.
exported_at	Timestamp	System-generated UTC export timestamp; not null.

8.2 Entity Relationships

1. TextPrompt to StrokeSequence (1 : N): One input prompt can generate multiple stroke lines or style variations (1:N).

2. WriterProfile to StrokeSequence (1 : N): A single writer profile conditions multiple stroke sequences across prompts (1:N).

3. StrokeSequence to QuantizedTokenSequence (1 : 1): Each continuous trajectory maps to exactly one discrete tokenized sequence (1:1).

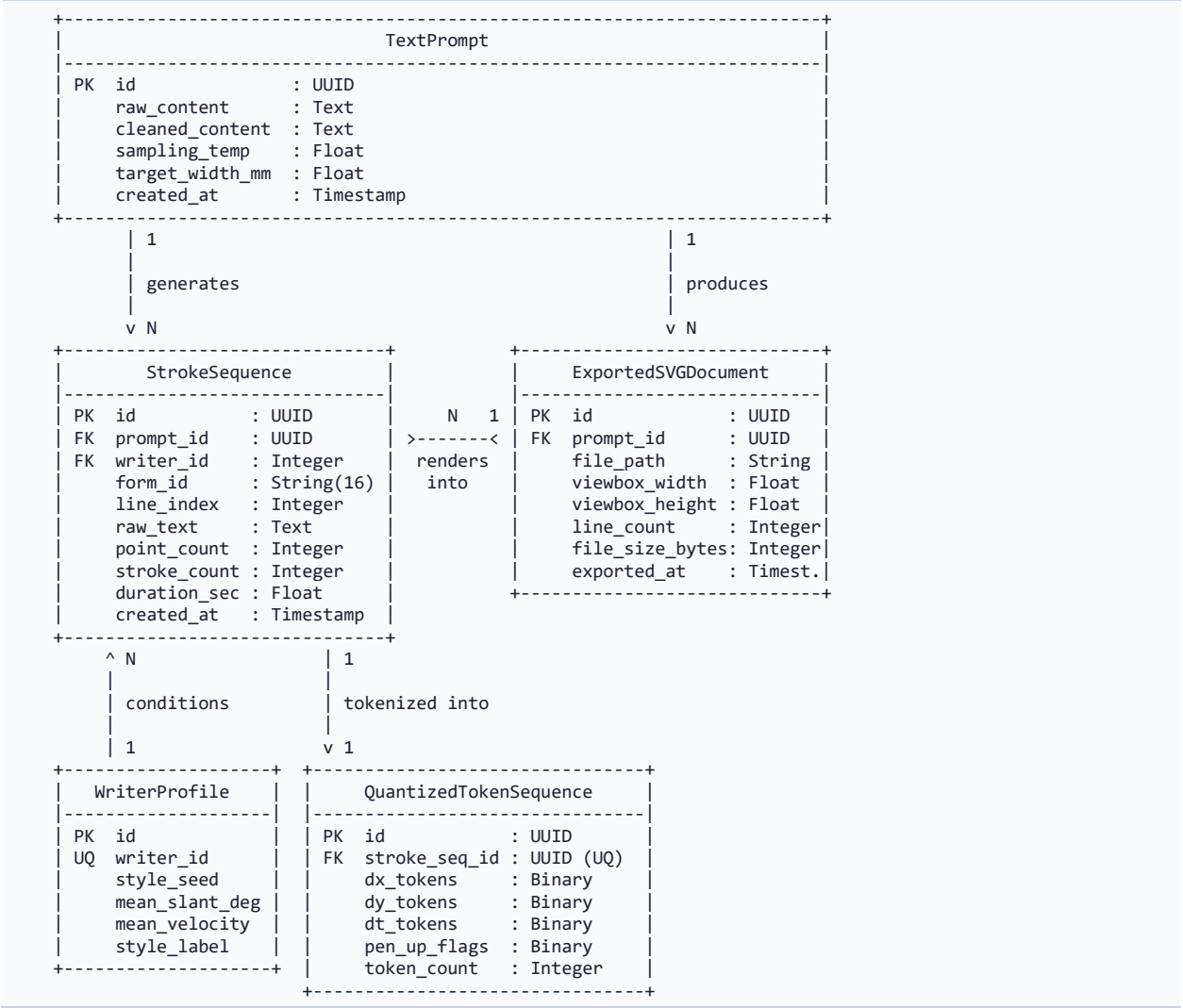
4. TextPrompt to ExportedSVGDocument (1 : N): A user prompt can be exported into multiple document iterations or styles (1:N).

5. StrokeSequence to ExportedSVGDocument (N : 1): An exported SVG document aggregates one or more line stroke sequences (N : 1).

8.3 Complete Entity-Relationship (ER) Diagram

Visual architectural layout, primary keys (PK), foreign keys (FK), and cardinalities across all entities:

Figure 2: Complete Entity-Relationship (ER) Architectural Diagram



9. Design Decisions and Alternatives Considered

#	Decision	Alternative rejected	Reason
D1	Discrete dual-mode tokenization (linear moves + asinh jumps).	Continuous Mixture Density Networks (MDNs).	MDNs suffer from variance collapse (sigma -> 0) causing NaN gradients and mode collapse. Discretization reformulates generation as Cross-Entropy, achieving zero NaN crashes and median drift of 0.032 x-height.
D2	Ragged 1D HDF5 storage with slice offset indices.	Fixed-length zero-padded 2D arrays.	Padding sequences to maximum length (857 points) wastes ~40% disk space and bakes a hard limit. Ragged storage saves 22 MB per split and enables dynamic batch collation.
D3	Direct SVG vector path export ('<path d=...>')	Raster PNG image generation via GANs or Diffusion.	Raster generation fixes resolution and causes pixelation. SVG paths are resolution-independent, styleable in CSS, and map directly to CNC pen plotters.

10. Acceptance Criteria

AC-1 (Live Generation): CLI accepts 'Calligrapher AI' and outputs valid, legible SVG stroke file within 1.5s.

AC-2 (Dataset Pipeline): Running 'prepare_data.py' compiles 12,178 lines and outputs 'train.h5', 'val.h5', 'test.h5' without errors.

AC-3 (Error Handling): Input with unsupported char ('Hello ~ World') halts immediately with clear positional error message.

AC-4 (Style Modulation): Varying temperature (0.5 to 1.5) modulates stroke curvature from stiff lettering to expressive cursive.

11. Out of Scope for This Release

1. Interactive Stylus Input: Live touch drawing canvas for user stroke capture; release focuses strictly on text-to-stroke.

2. Non-Latin Language Scripts: Multilingual non-Latin script synthesis; IAM training corpus is strictly English Latin.

3. Biometric Verification: Few-shot online biometric signature verification; identity authentication is a separate problem.

Institutional Evaluation & Faculty Sign-Off Block

Candidate Details	Faculty Supervision	Project Review Committee
Name: Agriim Rastogi Roll No.: 22515500004 Section: ND Signature: _____ Date: 11-09-2026	Palash Sir / Vansh Sir Faculty Mentors Department of CSE Signature: _____ Date: 11-09-2026	Gautam Mukherjee Project Reviewer GLA University, Greater Noida Signature: _____ Date: 11-09-2026